

Summary

C Programming: Advanced Concepts and Applications of Recursion

This document presents a technical synthesis of recursion in C programming, based on Lecture 25 by Pankaj Sharma. It explains the fundamental structure of recursive functions, methods for tracing recursive execution, and the behavioral differences caused by execution order and static variables. The summary also covers common recursive problem patterns used in competitive programming, including power calculation, binary bit manipulation, and branching recursion. Emphasis is placed on understanding execution flow through the stack and recursion tree models.

Key Takeaways

- Recursion requires a base case and a recursive step.
- Recursive calls are managed using the program stack (LIFO).
- Statement placement before or after the recursive call changes output order.
- Static variables maintain a single shared instance across recursive calls.
- Recursion is widely used in problems involving power calculation, binary representation, and branching logic.

1. Fundamentals of Recursive Implementation

Recursion occurs when a function calls itself to solve smaller versions of a problem. A valid recursive function must include two mandatory components:

1.1 Base Case (Small Input)

- Defines the termination condition.
- Handles the smallest input where the result is known directly.
- Prevents infinite recursion.

Example:

$$a^0 = 1$$

1.2 Recursive Step (Large Input)

- Applies when the problem cannot be solved directly.
- Performs minimal work and delegates the remaining task to another recursive call.

2. Mathematical Application: Power Function (a^b)

The calculation of a raised to the power b ($a > 0$, $b \geq 0$) can be implemented recursively.

Recursive Definition

- **Base Case:**

If $b = 0$, return 1

- **Recursive Step:**

return $a \times \text{power}(a, b - 1)$

Execution Mechanism

- Each function call creates an activation record on the stack.
- Every call waits for the next call to return a value.
- Multiplication is performed during stack unwinding.

Example:

5^3

→ $5 \times \text{power}(5,2)$

→ $5 \times \text{power}(5,1)$

→ $5 \times \text{power}(5,0)$

→ 1

→ 5

→ 25

→ 125

3. Tracing Methodologies

Tracing helps predict the output of recursive functions. Two primary techniques are used:

3.1 Stack Method (Activation Records)

- Each recursive call creates a separate activation record.
- Local variables and parameters are stored independently.
- Execution pauses at the recursive call until the child function returns.
- Uses Last-In, First-Out (LIFO) order.

3.2 Recursion Tree Method

- Each function call is represented as a node.
- Child nodes represent recursive calls.
- Traced from top to bottom and left to right.
- Especially useful for branching recursion (multiple recursive calls).

4. Execution Flow: Pre-Call vs. Post-Call

The position of statements relative to the recursive call changes execution behavior.

Execution Comparison

Statement Placement	Execution Order	Logic
Before Recursive Call	Same as call order	Executes during forward phase
After Recursive Call	Reverse of call order	Executes during backtracking phase

Illustrative Example

- **Pre-call print:**

f(4) → prints: 4 3 2 1

- **Post-call print:**

f(4) → prints: 1 2 3 4

Explanation:

- Post-call statements remain stored in the stack.
- They execute only when the base case is reached and the stack unwinds.

5. Static Variables in Recursion

Static variables exhibit distinct behavior in recursive functions.

Characteristics

- Only one copy exists in memory.
- Shared across all recursive calls.
- Initialization executes only once.
- Value changes persist across calls.

Example:

```
static int i = 5;
```

Impact on Output

- If a static variable is modified during the forward phase,
- Post-call prints display the final value of the variable,
- Not the value at the time of individual calls (often 0).

6. Technical Case Studies and Problem Solving

6.1 Binary Conversion and Bit Manipulation

Recursive logic using modulo and division:

- $n \% 2$ extracts the last binary bit.
- $n / 2$ processes remaining bits.

Function logic:

$$(n \% 2) + f(n / 2)$$

Purpose:

- Counts the number of set bits (1s).

Example:

Input: 513

Binary: 1000000001

Output: 2

6.2 Branching Recursion (f(12) Problem)

- A function may call itself multiple times in one return statement.

Example pattern:

$$f(x - 4) + f(x - 1) + 1$$

Approach:

- Compute smaller values first:

$$f(0), f(1), f(2), f(3)$$

- Use results to compute larger inputs.

Final result:

$$f(12) = 51$$

6.3 Conditional Logic Using $n \% 2$

In C, numerical values are treated as boolean values.

- $n \% 2 == 0 \rightarrow \text{Even} \rightarrow \text{False}$
- $n \% 2 == 1 \rightarrow \text{Odd} \rightarrow \text{True}$